

Perk the Star

Defend me, defend me! – Oa ka Perk!.

A Single-Player Orbital Tower Defense Game

Proposal 4

Project Proposal

Developers: Geo Ceff Gabaisen, Dexter Juevesano

Program / Year: BS Computer Science – I

Course & Section: CMSC 21 – A

Instructor: Ryan Ciriaco Dulaca

Executive Summary

Perk the Star is a single-player, real-time orbital tower defense game developed in C++ using the SFML library. The player commands the *Sol Defense Corps*, protecting the Sun from *Astrophage* — a photosynthetic microorganism of extraterrestrial origin feeding on stellar energy. The premise is drawn from *Project Hail Mary* (Andy Weir, 2021).

Players deploy orbital defense satellites across four concentric rings surrounding the Sun, managing a luminosity resource across twelve escalating waves. The central innovation is **orbital placement**: towers orbit the Sun at physics-derived angular velocities, creating a dynamic targeting puzzle. The Sun itself reacts to damage through four pixel-art expression states, serving as both health indicator and emotional core.

Narrative

Premise

Seventeen months ago, astronomers detected a 0.3% anomaly in the Sun’s luminosity. Six months ago, it was 2.1%. This morning: 4.8% and accelerating. The cause is *Astrophage* — a microorganism absorbing stellar radiation at the corona, reproducing on harvested energy, and spreading outward. Left unchecked, the Sun goes dark within decades.

You are the *Sol Commander*. You have twelve waves. You have a luminosity budget. *Spend it wisely.*

Setting & Entities

The game takes place in the orbital space around the Sun, rendered top-down. Four concentric orbital rings extend outward to ≈ 1.5 solar radii. *Astrophage* drift inward as bioluminescent organisms trailing light trails.

Entity	Description
The Sun	Protected entity and emotional anchor. Four pixel-art expression states: <i>Happy</i> (>80%), <i>Concerned</i> (60–80%), <i>Distressed</i> (30–60%), <i>Critical</i> (<30%). Fires solar flares every three waves.
Sol Commander	The player. Places towers, triggers flares, and manages Sol Credits between waves.
Astrophage	Enemy collective. Four biologically distinct variants introduced progressively across twelve waves.
Astrophage Prime	Wave 12 boss. Cycles through Shell, Active, and Frenzy phases; spawns Drifters continuously.

The Vision [WOW siya!]



Figure 1: Cute na Perk </3

Gameplay Systems

Core Loop

Each wave follows this structure: **(1)** Between-Wave Phase begins — players place and upgrade towers using Sol Credits. **(2)** Wave Active begins — Astrophage spawn from the outer edge and move inward. **(3)** Towers orbit their rings, firing at any Astrophage within their engagement arc. **(4)** Astrophage that reach the corona deal luminosity damage. **(5)** On wave clear, Sol Credits are awarded and the game checks win/loss conditions.

Key Statistics

Stat	Range	Description
Luminosity	0–100%	The Sun’s health. Reduced by Astrophage breaches. Cannot be restored. Reaching 0% ends the run.
Sol Credits	0–∞	Economy resource. Earned by defeating enemies and surviving waves. Spent on tower placement and upgrades.
Flare Charge	0–1	Accumulates every 3 waves. On trigger, fires a radial burst clearing 25% of active Astrophage.
Performance Score	0–∞	Tracks decision quality; determines final rank: <i>Dim</i> , <i>Bright</i> , or <i>Full Shine</i> .

Tower System

Towers fire within a fixed **engagement arc** of $\pm 30^\circ$ relative to their orbital heading. Inner rings sweep past enemies quickly but with short engagement windows; outer rings orbit slowly with longer windows. This asymmetry is the core strategic tension.

Tower	Orbit	Tier	Special Behavior
Photon Splitter	Inner	1	Fast-firing beam of reflected sunlight. Effective against Drifters and Blooms in high-frequency Ring 1 sweeps.
Cryo Probe	Mid	1	Cold-plasma suppression field; reduces Astrophage speed by 40%. Pure support; deals no damage.
Bio-Lab Station	Outer	2	Analyzes one variant over 15 seconds; then deals $3\times$ damage to it for the rest of the wave. Required against Mimics and Prime.
Magnetic Net	Any	2	Clusters nearby Astrophage together; clustered enemies share all incoming damage. 12-second cooldown.
Helios Cannon	Inner	3	Redirects a solar flare into a targeted stellar column. Costs 30 Sol Credits. Devastating into a Magnetic Net cluster.
Tardigrade Bomb	Mid	3	Engineered organisms that chain-infect up to 5 adjacent Astrophage. Slow projectile; must lead dense clusters.

Astrophage Variants

Variant	HP	Speed	Special Behavior
Drifter	Low	Medium	Baseline unit. Straight-line approach. Appears in all twelve waves.
Bloom	Medium	Medium	Splits into three Drifters on direct-damage death. Requires AoE or cryo-debuff first.
Coronal Burrower	High	Slow	Burrows into the Sun on arrival, draining -2% luminosity/sec until excavated by a Bio-Lab Station. Always priority.
Photon Mimic	Medium	Medium	Invisible to the Photon Splitter. Only Bio-Lab Station and Cryo Probe can detect it. Forces tower diversity.
Solar Farmer	Scales	Increases	Absorbs energy from damage towers, gaining HP and speed per hit. Must be cryo-debuffed before engaging.
Astrophage Prime	Extreme	Slow	Wave 12 boss. <i>Shell</i> (immune; Bio-Lab breaks it) → <i>Active</i> (vulnerable) → <i>Frenzy</i> (double Drifter spawn).

Orbital Mechanics

Towers occupy one of 28 slots across four concentric rings. Angular velocities are derived from Kepler's Third Law: $T^2 \propto r^3$.

Ring	Name	Radius (px)	Period (s)	Slots	Best Towers
1	Corona Belt	80	6	4	Photon Splitter, Helios Cannon
2	Chromosphere Band	140	11	6	Cryo Probe, Tardigrade Bomb
3	Photosphere Arc	210	17	8	Bio-Lab Station, Magnetic Net
4	Outer Veil	290	26	10	Early intercept, scout-role towers

Tower screen position is updated each frame via polar-to-Cartesian conversion:

$$x = x_{\text{sun}} + r \cos \theta, \quad y = y_{\text{sun}} + r \sin \theta$$

where θ increments each tick by angular velocity $\omega = \frac{2\pi}{T}$.

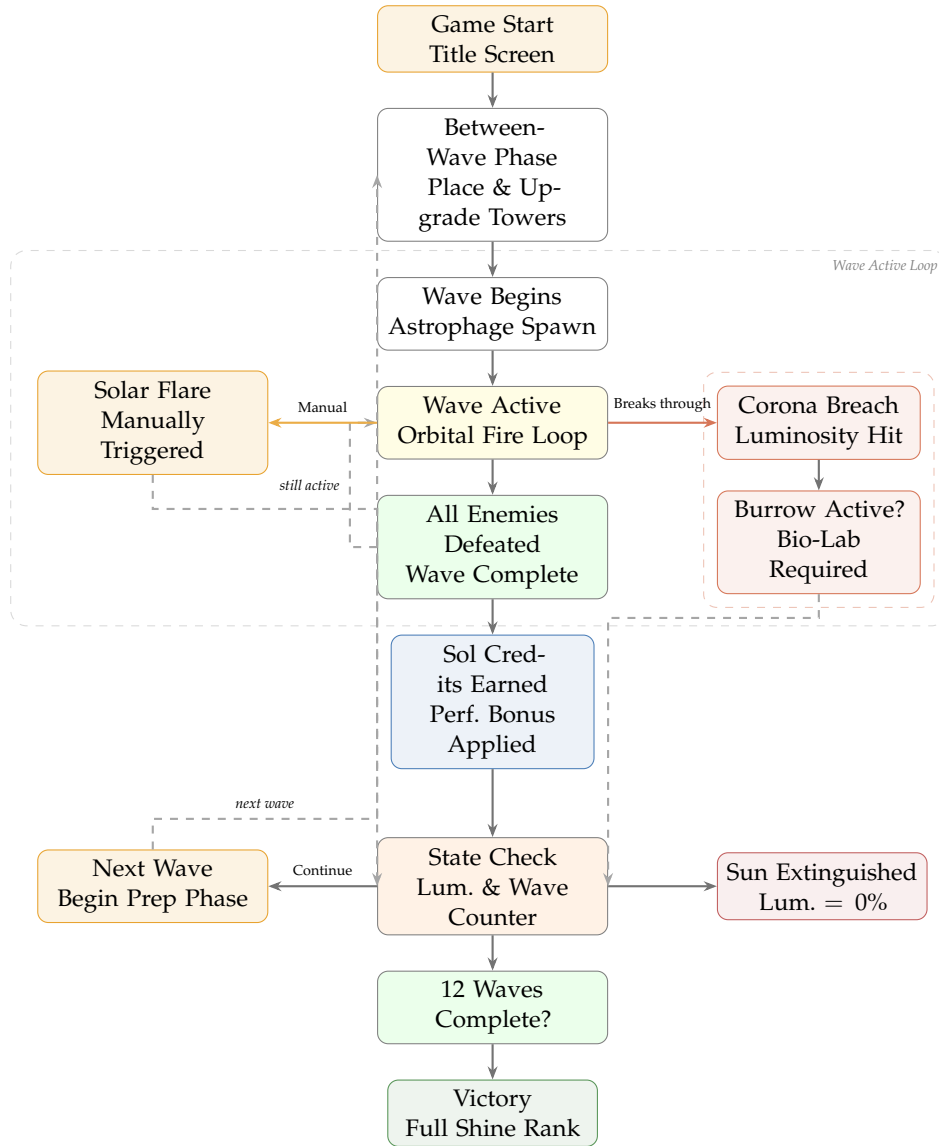
Every three waves, the player may trigger a manual solar flare: a radial burst in a 90-degree corona cone. Timing it into a Magnetic Net cluster can clear an entire screen segment.

Wave Progression

Twelve waves with progressive Astrophage introduction. Configurations are loaded from external JSON at runtime, allowing balance changes without recompilation.

Wave	Name	Variants	Notes
1	First Contact	Drifter	Tutorial. Introduces engagement arc mechanics.
2	The Vanguard	Drifter	Introduces Sol Credit economy and upgrades.
3	Splitting Point	Drifter, Bloom	Bloom introduced; direct fire alone is insufficient.
4	Going Under	Drifter, Bloom, Burrower	First Coronal Burrower; Bio-Lab becomes necessary.
5	Invisible Hand	Drifter, Mimic	Photon Mimic introduced; forces tower diversity.
6	Solar Storm	Drifter, Bloom, Mimic	<i>Event:</i> Mid-wave auto-flare. Cryo Probes disrupted 8s.
7	Night Side	All Tier 1–2	<i>Event:</i> Wave from behind the Sun; Ring 1–2 blind for 20s.
8	The Harvest	Drifter, Bloom, Farmer	Solar Farmer introduced. Cryo combination mandatory.
9	Feeding Frenzy	All (no Prime)	Dense mixed swarm; prime Magnetic Net + Cannon window.
10	Research Surge	All (no Prime)	<i>Event:</i> Bio-Lab runs at $4\times$ speed.
11	Penultimate	All (no Prime)	Maximum density; outer ring becomes critical.
12	Astrophage Prime	All + Prime	Boss wave. Prime cycles Shell $\rightarrow$ Active $\rightarrow$ Frenzy.

Core Gameplay Flow Diagram



Game Endings

Ending	Type	Trigger Condition
Full Shine	Good (S-Rank)	12 waves completed with luminosity above 80%.
Dim but Alive	Good (B-Rank)	12 waves completed with luminosity between 20–80%.
Last Light	Good (C-Rank)	12 waves completed with luminosity between 1–20%.
Sun Extinguished	Bad	Luminosity hits 0% at any point. Post-mortem screen shows full tower loadout, luminosity trend curve, and the exact variant that dealt the killing blow — presented as a clinical Sol Defense Corps incident report.

Data Structures and Implementation

All structures interface with SFML and follow a flat, component-oriented architecture. Orbital motion is computed analytically from ring parameters; no external physics engine is used.

`Tower / OrbitalTower` — stores tower type (enum), ring index, slot index, base damage, fire rate, and arc width. `OrbitalTower` adds ring radius, angular position θ , and angular velocity ω ; screen position is computed from the polar formula each frame. Engagement checks compare angular offsets within a defined threshold.

`Astrophage` — stores variant type, position, velocity, HP, and a speed modifier. Boolean flags handle burrowing and cloaking states. A defeated flag marks units for removal and reward processing.

`Sun` — tracks luminosity, expression state, animation phase, and flare charge. Provides methods for damage, flare trigger, and defeat check. Expression transitions are timed crossfades.

`Wave` — loaded from JSON via `nlohmann/json`. Stores wave index, spawn data, optional event modifiers, and reward thresholds, enabling balance changes without recompilation.

`GameState` — centralized runtime container. Stores current wave, player resources, score, active Sun reference, and lists of active towers and enemies. Current game phase tracked as an enum. All systems receive a reference to this struct.

Graphics and Visual Design

Perk the Star uses a constrained pixel art aesthetic: 4-pixel snap grid, all sprites at 16×16 or 32×32 base resolution, anti-aliasing disabled globally. The tone is *cozy crisis* — warm and cheerful at full luminosity, shifting to muted and desaturated as the Sun dims. The palette is anchored around deep navy for space, warm amber for the Sun and orbital rings, coral for *Astrophage*, and cool blue/teal for player technology.

Animations are frame-based sprite sheet sequences with no external animation library. Tower screen positions recalculate each frame from the polar formula. *Astrophage* pulse between two frames at 1.5 Hz baseline, accelerating to 3 Hz near the corona. Sun expressions crossfade over 0.4 seconds on luminosity threshold crossings. On *Sun Extinguished*, the face transitions to neutral over 3 seconds as the star field dims to black. [joke time!!! di diay mabuhat..</3]

References

- Weir, A. (2021). *Project Hail Mary*. Ballantine Books.
- SFML Documentation (2.6) — <https://www.sfm1-dev.org/documentation/2.6.0/>
- Lohmann, N. (2023). *nlohmann/json: JSON for Modern C++* (v3.11). GitHub.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- Meyers, S. (2014). *Effective Modern C++*. O'Reilly Media.
- Sama, N. (2024). *That's Not My Neighbor*. itch.io.